

Abstract

Abstract I

Research software repositories in monorepo configurations accumulate three categories of shared resources that individual projects must consume without re-implementing discovery logic: **data pools** (bibliographies, contacts, datasets), **governance rules** (style guides, coverage thresholds, citation schemas), and **executable tools** (code executors, validators, skill invocations). Without a canonical integration pattern, projects either duplicate discovery logic or silently ignore resources that fail to load — both outcomes degrade reproducibility and collaborative cohesion [@Wilson2014best; @Taschuk2017ten].

Abstract II

This paper presents `template_pools_rules_tools`, a meta-project exemplar that demonstrates how a single project can programmatically discover, validate, and exercise all three resource categories with zero tight coupling to any specific resource instance. The exemplar comprises four Python modules — `fonds_reader`, `rules_applier`, `tools_invoker`, and `integration` — plus three thin orchestration scripts and a fully token-injected manuscript pipeline.

The architecture (`@fig:architecture`) separates *resource ownership* from *resource consumption*. Resources live in top-level `fonds/`, `rules/`, and `tools/` directories and are never modified by consumers. Each resource exposes a typed manifest (`fonds.yaml`, `rules.yaml`, `tools.yaml`) that the corresponding reader module uses for discovery and validation. All readers implement graceful fallbacks: they return `None` or empty collections when a resource is absent, log a warning via the standard library logging module, and allow the integration pipeline to continue.

Abstract III

In a representative pipeline run, the integration demo loaded `{{integration.fonds_loaded}}` funds, validated `{{integration.rules_sets_ok}}` rule sets, discovered `{{integration.tools_discovered}}` tools, and processed `{{integration.bib_entries}}` bibliography entries — all reported as structured JSON that populates manuscript variable tokens at render time. Tests covering the four `src/` modules achieve 90% line coverage and use real file paths rather than mocks, ensuring that reported counts are genuine.

The `template_pools_rules_tools` exemplar provides a reference implementation that any project in the template repository can consult when designing its own resource-consumption layer.

Keywords: research software engineering, monorepo architecture, reproducibility, funds, governance rules, tool discovery, graceful degradation